

Sobreposição de computação e comunicação

Eugenio Vitor Lopes dos Santos

DCA3703, Programação Paralela, Universidade Federal do Rio Grande do Norte

1 Enunciado

Simular a difusão de calor em uma barra 1D distribuída entre dois ou mais processos MPI, com células extras para trocar as bordas. Comparar três versões: comunicação com `MPI_Send` e `MPI_Recv`; comunicação não bloqueante com `MPI_Isend`, `MPI_Irecv` e `MPI_Wait`; e uma versão que usa `MPI_Test` enquanto calcula os pontos internos.

2 Fundamentação

Cada processo armazena um trecho da barra e duas células fantasma, uma de cada lado. A atualização de uma célula usa seu valor e os valores das duas vizinhas no passo anterior:

$$u_i^{n+1} = u_i^n + 0,2 (u_{i-1}^n - 2u_i^n + u_{i+1}^n).$$

As células das extremidades locais dependem dos valores recebidos dos processos vizinhos. As células estritamente internas não dependem dessa troca e podem ser atualizadas enquanto ela está pendente. Nas extremidades globais da barra, as células fantasma permanecem em zero.

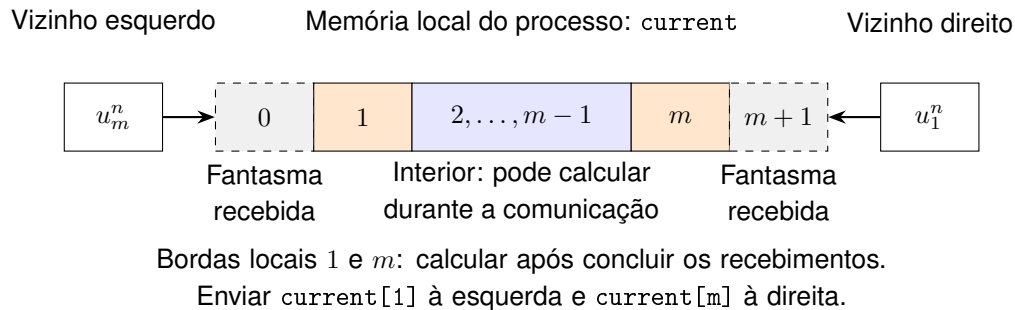


Figura 1: Índices locais, com $m = \text{count}$. As setas mostram os recebimentos de um processo com dois vizinhos; cada mensagem contém um `double`. Nas extremidades globais, o vizinho ausente é `MPI_PROC_NULL` e a célula fantasma permanece em zero.

Na Figura 1, atualizar $i = 1$ exige ler `current[0]`, enquanto atualizar $i = m$ exige ler `current[m+1]`. Para $2 \leq i \leq m-1$, os três valores necessários já pertencem ao processo. Todas as atualizações leem `current` e escrevem em `next`; assim, nenhum valor do passo n é substituído por um valor do passo $n+1$ durante o cálculo.

2.1 Troca bloqueante

`MPI_Send` e `MPI_Recv` trocam as bordas antes de qualquer atualização. O código envia primeiro à direita e recebe da esquerda; em seguida, envia à esquerda e recebe da direita. `MPI_PROC_NULL` representa o vizinho ausente nas extremidades globais.

2.2 Comunicação ponto a ponto não bloqueante

As operações não bloqueantes retornam sem exigir que a transferência esteja concluída. Enquanto um recebimento estiver pendente, as células fantasma correspondentes não podem ser usadas no cálculo.

2.2.1 Escondendo latência com sobreposição de computação e comunicação

Após iniciar a troca, o processo pode atualizar as células internas, que não dependem das bordas recebidas. Só depois da conclusão dos recebimentos atualiza as duas extremidades locais. A versão com `MPI_Wait` espera antes de calcular; a versão com `MPI_Test` consulta os recebimentos durante o cálculo interno. Essa consulta também tem custo.

2.2.2 Criação de `MPI_Request`

- `MPI_Isend`: inicia o envio de uma borda e devolve um `MPI_Request` para acompanhar a operação.
- `MPI_Irecv`: inicia o recebimento de uma borda e devolve um `MPI_Request` para acompanhar a operação.

2.2.3 Consumo de `MPI_Request`

- `MPI_Wait`: espera a conclusão da operação associada ao `MPI_Request`.
- `MPI_Test`: verifica se a operação terminou sem bloquear; se ainda estiver pendente, o pedido continua válido.

2.3 Ordem das operações em cada passo

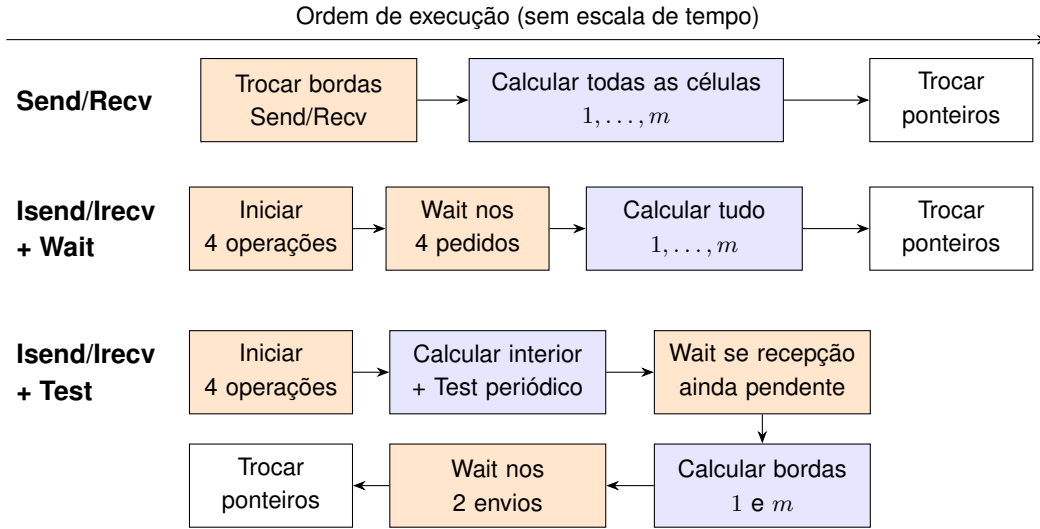


Figura 2: Sequência implementada pelas três versões. Na terceira, seguir as setas até a linha inferior. O cálculo interno oferece uma oportunidade de sobreposição; o diagrama não representa durações medidas nem garante progresso simultâneo das transferências.

Na versão com `MPI_Wait`, os quatro pedidos são concluídos antes da primeira atualização. Na versão com `MPI_Test`, o laço calcula $i = 2, \dots, m - 1$ e consulta os recebimentos ainda pendentes quando $i \% 1024 == 0$. O teste informa a conclusão por uma flag; se ela continuar falsa ao terminar o interior, `MPI_Wait` garante que a célula fantasma esteja pronta antes do cálculo das bordas.

Os pedidos `requests[0]` e `requests[1]` acompanham os recebimentos; os pedidos `requests[2]` e `requests[3]`, os envios. Enquanto um envio estiver pendente, seu buffer em `current` não pode ser alterado. Escrever os resultados em `next` permite calcular sem modificar esse buffer. A espera pelos envios ao final libera sua reutilização nos passos seguintes; só então o programa troca os ponteiros dos dois vetores.

3 Experimento

Foram executados cinco jobs no NPAD, na partição `intel-128`: 2116320, 2116321, 2116322, 2116323 e 2116324. Cada job usou um processo MPI por nó e uma CPU por processo, com, respectivamente, 2, 4, 8, 10 e 12 processos e nós. A barra permaneceu fixa em 1 048 576 células, portanto o número de células locais variou de 524 288 a 87 381. O programa foi compilado com `mpicc -std=c11 -O3 -Wall -Wextra` e executou 1 000 passos com coeficiente 0,2.

A metade esquerda começou com temperatura 1 e a metade direita com 0. Em cada job foram feitas dez execuções do programa. Em cada execução, as três versões foram medidas nessa ordem e com a mesma condição inicial. O intervalo de `MPI_Wtime` abrange os passos da simulação e a barreira final; não inclui inicialização do MPI, compilação, alocação, inicialização da barra nem

redução da soma final. As saídas registraram avisos de inicialização do dispositivo OpenFabrics; nenhum transporte foi explicitamente excluído nesta medição.

A variação de processos e nós foi usada para observar o comportamento das trocas de halo em diferentes quantidades de trabalho local. O foco da análise é a comparação direta entre as três estratégias de comunicação MPI.

4 Resultados

Processos/nós	Send/Recv (s)	Isend/Irecv+Wait (s)	Isend/Irecv+Test (s)
2	0,601704	0,595326	0,914577
4	0,341695	0,320561	0,452932
8	0,168256	0,158681	0,233792
10	0,141879	0,132283	0,188951
12	0,125474	0,107324	0,155934

Tabela 1: Medianas das dez execuções por configuração. Como foi usado um processo por nó, a coluna também indica a quantidade de nós. A soma final foi 524 272,535328 em todas as versões.

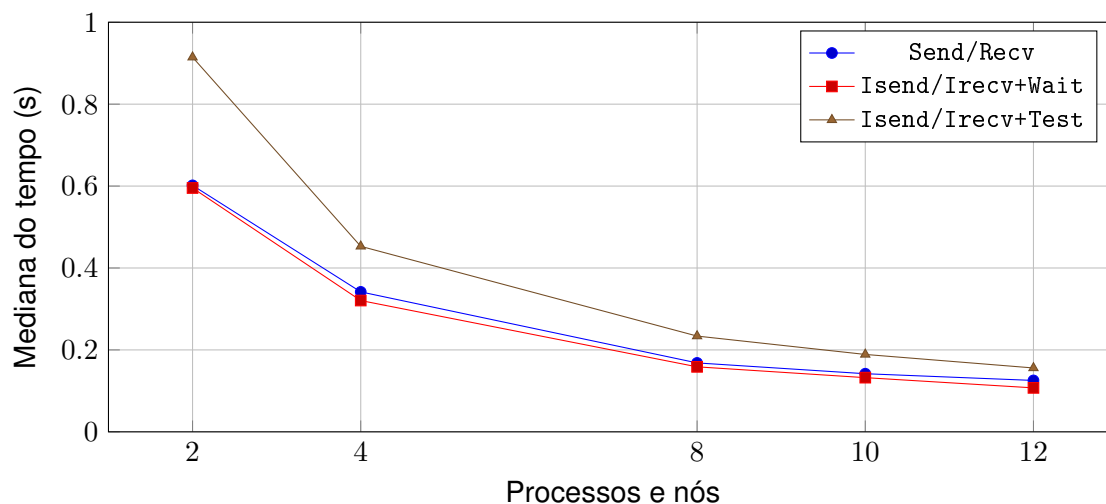


Figura 3: Medianas das dez execuções para cada configuração de processos e nós.

A versão Isend/Irecv+Wait apresentou o menor tempo mediano em todas as configurações. A versão bloqueante ficou muito próxima dela, enquanto a versão com MPI_Test permaneceu mais lenta. Portanto, neste experimento, iniciar as operações não bloqueantes e esperar antes do cálculo foi mais favorável que consultar periodicamente os recebimentos durante o cálculo interno.

A versão com MPI_Test cria uma oportunidade de sobreposição, mas essa oportunidade não trouxe ganho observado. Cada troca de halo contém apenas um `double` por direção, de modo que a latência e o custo das operações de comunicação têm importância relativa. As consultas

periódicas também introduzem trabalho adicional; o experimento não isolou esses custos nem mediu o progresso interno do MPI.

Quando a quantidade de processos e nós aumenta, o tempo total diminui porque cada processo calcula menos células. Essa tendência não deve ser interpretada isoladamente como melhoria da comunicação: a configuração altera simultaneamente a quantidade de trabalho local e o número de vizinhos envolvidos. Com um processo por nó, as trocas entre vizinhos usam a rede entre nós; assim, os resultados servem para comparar as estratégias sob diferentes granulosidades de trabalho, não para separar o efeito exclusivo do número de nós.

A soma final idêntica em todas as execuções é uma verificação global de consistência, não uma comparação célula a célula. Os avisos OpenFabrics caracterizam o ambiente de execução; como a inicialização do MPI está fora do intervalo cronometrado, o experimento não isolou eventual influência desses diagnósticos sobre o desempenho medido.

Anexo A: Código completo

```
1 #include <mpi.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 #define CELLS 1048576
6 #define STEPS 1000
7 #define COEFFICIENT 0.2
8
9 static void update(double *next, const double *current, int i) {
10     next[i] = current[i] + COEFFICIENT *(current[i - 1] - 2.0 * current[i] +
11     current[i + 1]);
12 }
13
14 static void blocking(double *next, double *current, int count, int left, int right)
15 {
16     MPI_Send(&current[count], 1, MPI_DOUBLE, right, 0, MPI_COMM_WORLD);
17     MPI_Recv(&current[0], 1, MPI_DOUBLE, left, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE
18     );
19     MPI_Send(&current[1], 1, MPI_DOUBLE, left, 1, MPI_COMM_WORLD);
20     MPI_Recv(&current[count + 1], 1, MPI_DOUBLE, right, 1, MPI_COMM_WORLD,
21     MPI_STATUS_IGNORE);
22
23     for (int i = 1; i <= count; ++i) update(next, current, i);
24 }
25
26 static void nonblocking(double *next, double *current, int count, int left, int
27 right) {
28     MPI_Request requests[4];
29     MPI_Irecv(&current[0], 1, MPI_DOUBLE, left, 0, MPI_COMM_WORLD, &requests[0]);
30     MPI_Irecv(&current[count + 1], 1, MPI_DOUBLE, right, 1, MPI_COMM_WORLD, &
31     requests[1]);
32     MPI_Isend(&current[count], 1, MPI_DOUBLE, right, 0, MPI_COMM_WORLD, &requests
33     [2]);
34     MPI_Isend(&current[1], 1, MPI_DOUBLE, left, 1, MPI_COMM_WORLD, &requests[3]);
35     for (int i = 0; i < 4; ++i) MPI_Wait(&requests[i], MPI_STATUS_IGNORE);
36
37     for (int i = 1; i <= count; ++i) update(next, current, i);
38 }
39
40 static void overlap(double *next, double *current, int count, int left, int right)
41 {
42     MPI_Request requests[4];
43     MPI_Irecv(&current[0], 1, MPI_DOUBLE, left, 0, MPI_COMM_WORLD, &requests[0]);
44     MPI_Irecv(&current[count + 1], 1, MPI_DOUBLE, right, 1, MPI_COMM_WORLD, &
45     requests[1]);
46     MPI_Isend(&current[count], 1, MPI_DOUBLE, right, 0, MPI_COMM_WORLD, &requests
47     [2]);
48     MPI_Isend(&current[1], 1, MPI_DOUBLE, left, 1, MPI_COMM_WORLD, &requests[3]);
49
50     int received_left = 0, received_right = 0;
51     for (int i = 2; i < count; ++i) {
```

```

42     update(next, current, i);
43     if (i % 1024 == 0) {
44         if (!received_left)
45             MPI_Test(&requests[0], &received_left, MPI_STATUS_IGNORE);
46         if (!received_right)
47             MPI_Test(&requests[1], &received_right, MPI_STATUS_IGNORE);
48     }
49 }
50 if (!received_left) MPI_Wait(&requests[0], MPI_STATUS_IGNORE);
51 if (!received_right) MPI_Wait(&requests[1], MPI_STATUS_IGNORE);
52 update(next, current, 1);
53 update(next, current, count);
54 MPI_Wait(&requests[2], MPI_STATUS_IGNORE);
55 MPI_Wait(&requests[3], MPI_STATUS_IGNORE);
56 }
57
58 int main(int argc, char **argv) {
59     int rank, process_count;
60     MPI_Init(&argc, &argv);
61     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
62     MPI_Comm_size(MPI_COMM_WORLD, &process_count);
63     if (process_count < 2 || process_count > CELLS / 2) {
64         if (rank == 0) fprintf(stderr, "Use de 2 a %d processos.\n", CELLS / 2);
65         MPI_Finalize();
66         return EXIT_FAILURE;
67     }
68
69     int count = CELLS / process_count + (rank < CELLS % process_count);
70     int start = rank * (CELLS / process_count) + (rank < CELLS % process_count ?
        rank : CELLS % process_count);
71     int left = rank == 0 ? MPI_PROC_NULL : rank - 1;
72     int right = rank == process_count - 1 ? MPI_PROC_NULL : rank + 1;
73     double *a = calloc((size_t)count + 2, sizeof(double));
74     double *b = calloc((size_t)count + 2, sizeof(double));
75     if (!a || !b) {
76         fprintf(stderr, "Falha de alocacao no processo %d.\n", rank);
77         MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
78     }
79
80     void (*versions[])(double *, double *, int, int, int) = {
81         blocking, nonblocking, overlap
82     };
83     const char *names[] = {"Send/Recv", "Isend/Irecv+Wait", "Isend/Irecv+Test"};
84     if (rank == 0) printf("Versao\tTempo(s)\tSoma final\n");
85     for (int version = 0; version < 3; ++version) {
86         for (int i = 1; i <= count; ++i)
87             a[i] = start + i - 1 < CELLS / 2 ? 1.0 : 0.0;
88         a[0] = a[count + 1] = 0.0;
89         b[0] = b[count + 1] = 0.0;
90         double *current = a, *next = b;
91
92         MPI_Barrier(MPI_COMM_WORLD);
93         double begin = MPI_Wtime();

```

```

94     for (int step = 0; step < STEPS; ++step) {
95         versions[version](next, current, count, left, right);
96         double *temporary = current;
97         current = next;
98         next = temporary;
99     }
100     MPI_Barrier(MPI_COMM_WORLD);
101     double elapsed = MPI_Wtime() - begin;
102
103     double local_sum = 0.0, total_sum = 0.0;
104     for (int i = 1; i <= count; ++i) local_sum += current[i];
105     MPI_Reduce(&local_sum, &total_sum, 1, MPI_DOUBLE, MPI_SUM, 0,
106               MPI_COMM_WORLD);
107     if (rank == 0) printf("%s\t%.6f\t%.6f\n", names[version], elapsed,
108                           total_sum);
109 }
110
111 free(a);
112 free(b);
113 MPI_Finalize();
114 return EXIT_SUCCESS;
115 }

```

Anexo B: Script Slurm

```

1  #!/bin/bash
2  #SBATCH --job-name=tarefa15
3  #SBATCH --output=runs/%x-%j.out
4  #SBATCH --nodes=2
5  #SBATCH --ntasks=2
6  #SBATCH --ntasks-per-node=1
7  #SBATCH --cpus-per-task=1
8  #SBATCH --time=00:10:00
9  #SBATCH --partition=intel-128
10
11  set -euo pipefail
12
13  echo "Job ID: $SLURM_JOB_ID"
14  echo "Processos: $SLURM_NTASKS"
15  echo "Nos: $SLURM_JOB_NUM_NODES"
16  echo "Lista de nos:"
17  scontrol show hostnames "$SLURM_JOB_NODELIST"
18
19  binary="tarefa15-${SLURM_JOB_ID}.bin"
20  trap 'rm -f "$binary"' EXIT
21
22  mpicc -std=c11 -O3 -Wall -Wextra -o "$binary" tarefa15.c
23
24  for run in $(seq 1 10); do
25      echo "Execucao $run"
26      mpirun -np "$SLURM_NTASKS" "$binary"
27  done

```
